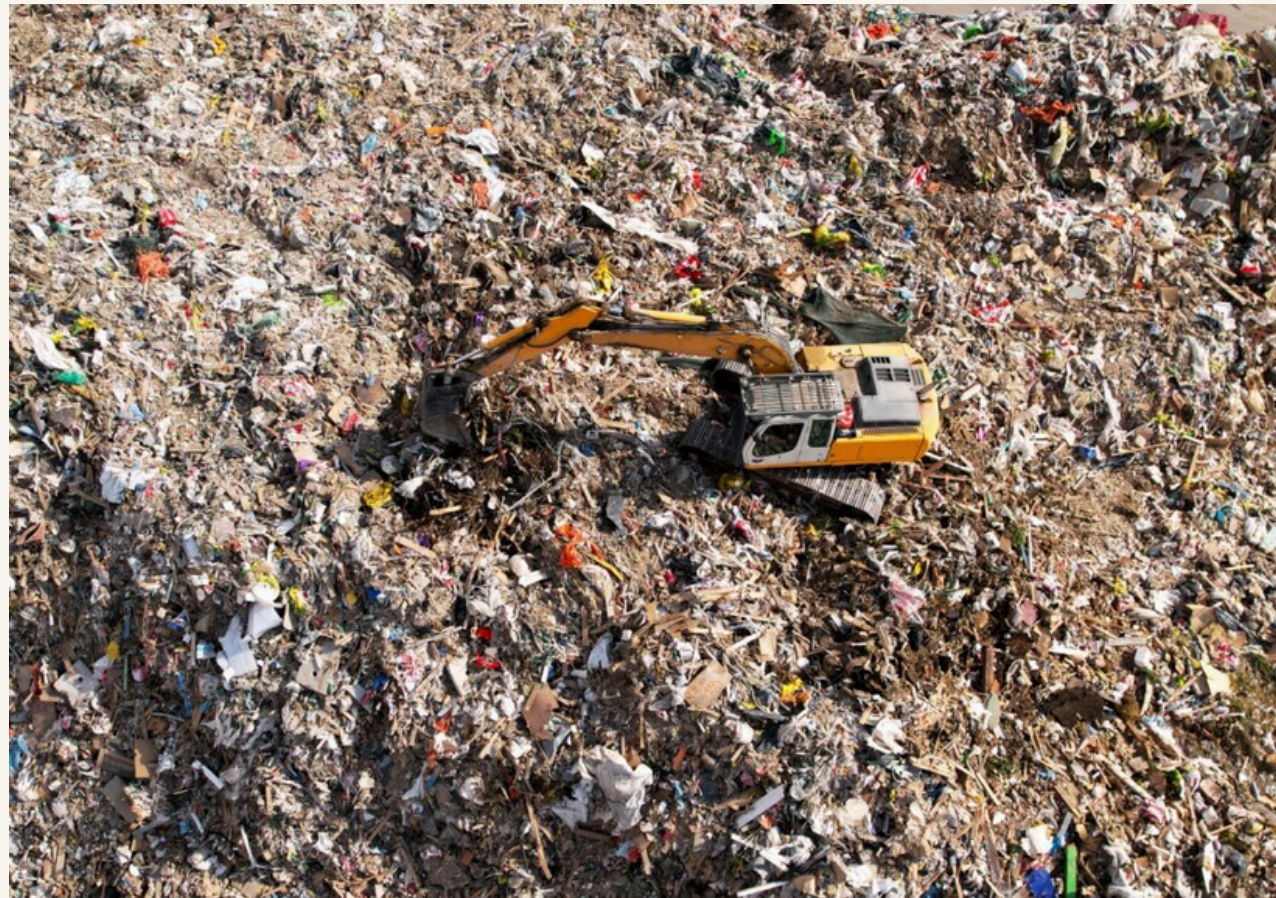


THE APPLICATION OF DEEP LEARNING TO AUTOMATED WASTE CATEGORISATION USING IMAGE CLASSIFICATION

Louis Vidal



THE WORLD HAS A WASTE PROBLEM



- **2 Billion Tons of waste produced globally each year**
- **Manual sorting is inefficient, expensive, and error-prone.**
- **Contamination ruins entire batches of recyclables.**



cardboard

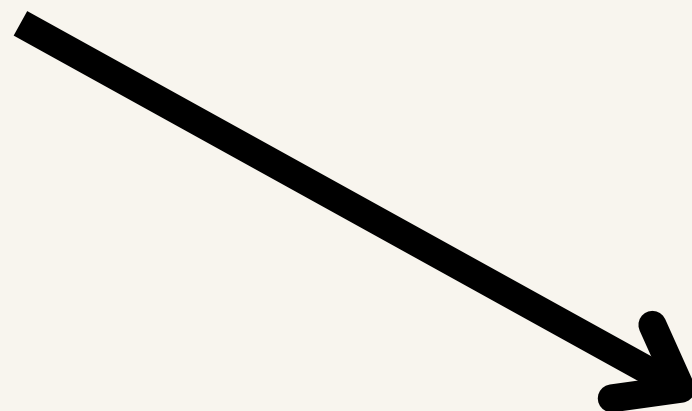
glass

metal

paper

plastic

trash



cardboard

glass

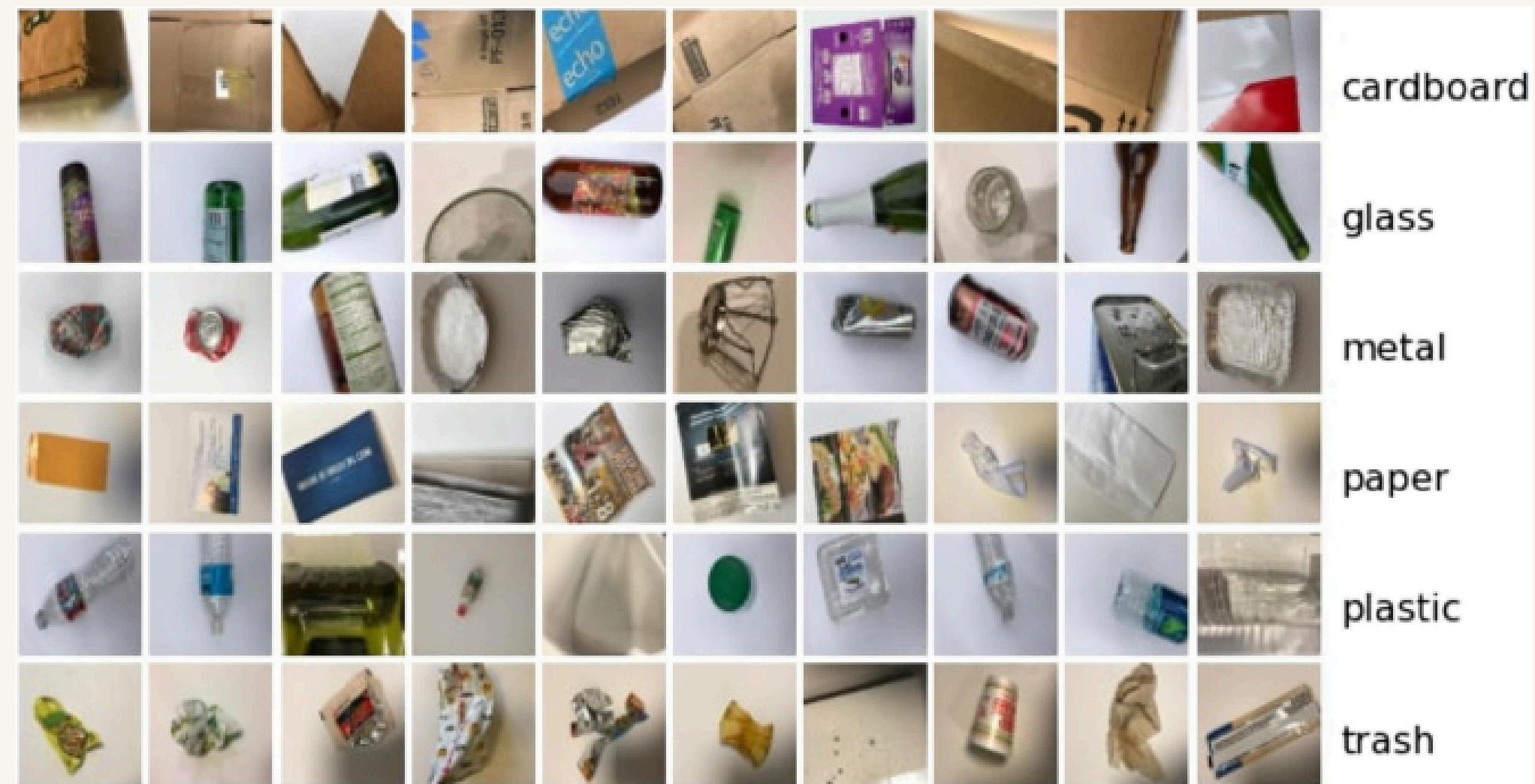
metal

paper

plastic

trash

THE DATASET



The Garbage Classification Dataset contains 6 classifications: cardboard (393), glass (491), metal (400), paper(584), plastic (472) and trash(127).

MY FIRST ATTEMPT

Class: glass



Class: cardboard



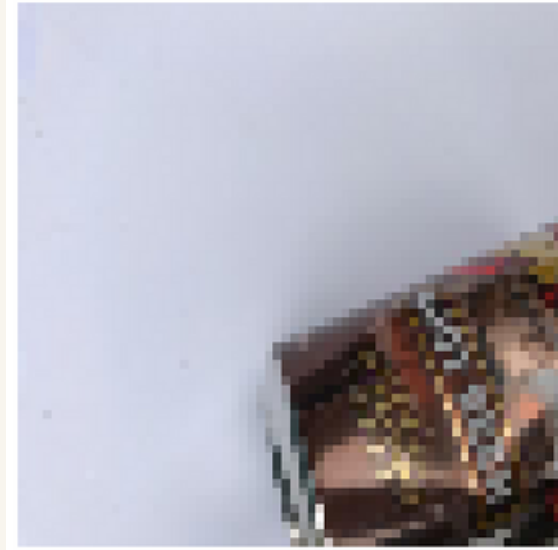
Class: trash



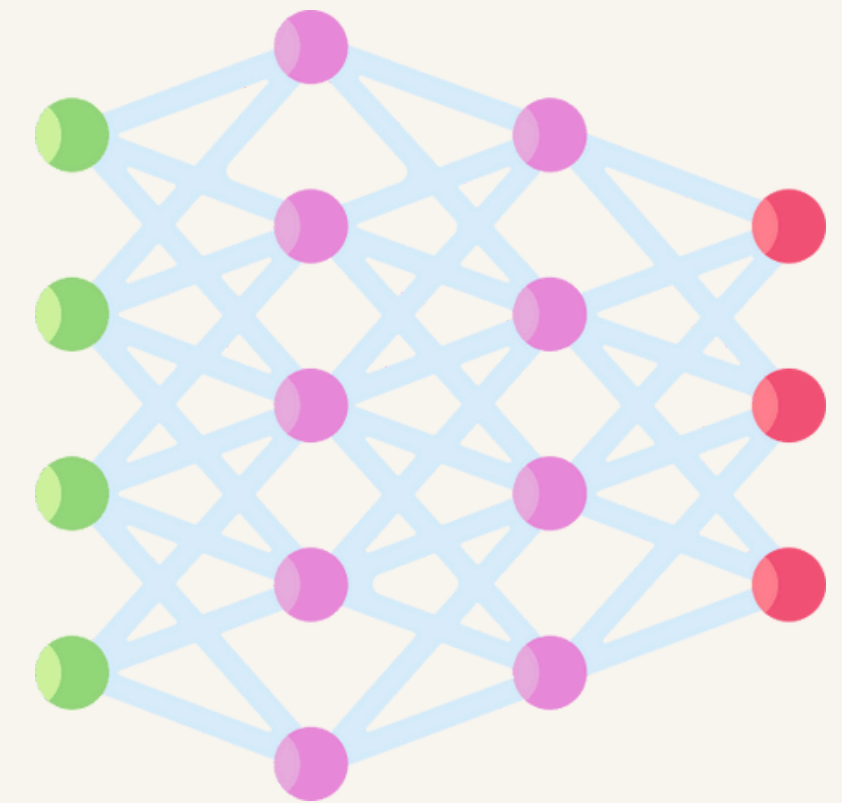
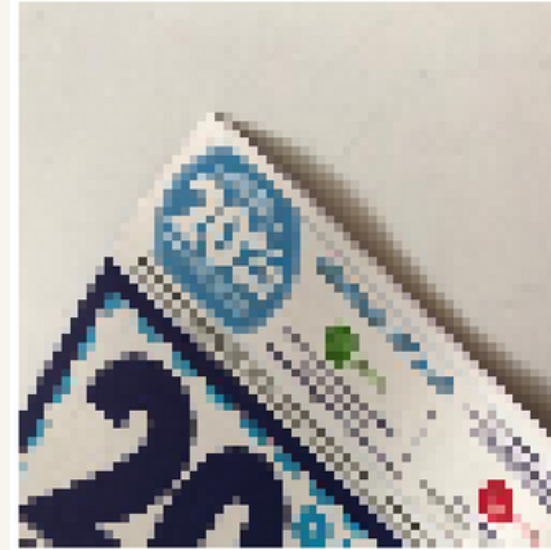
Class: plastic



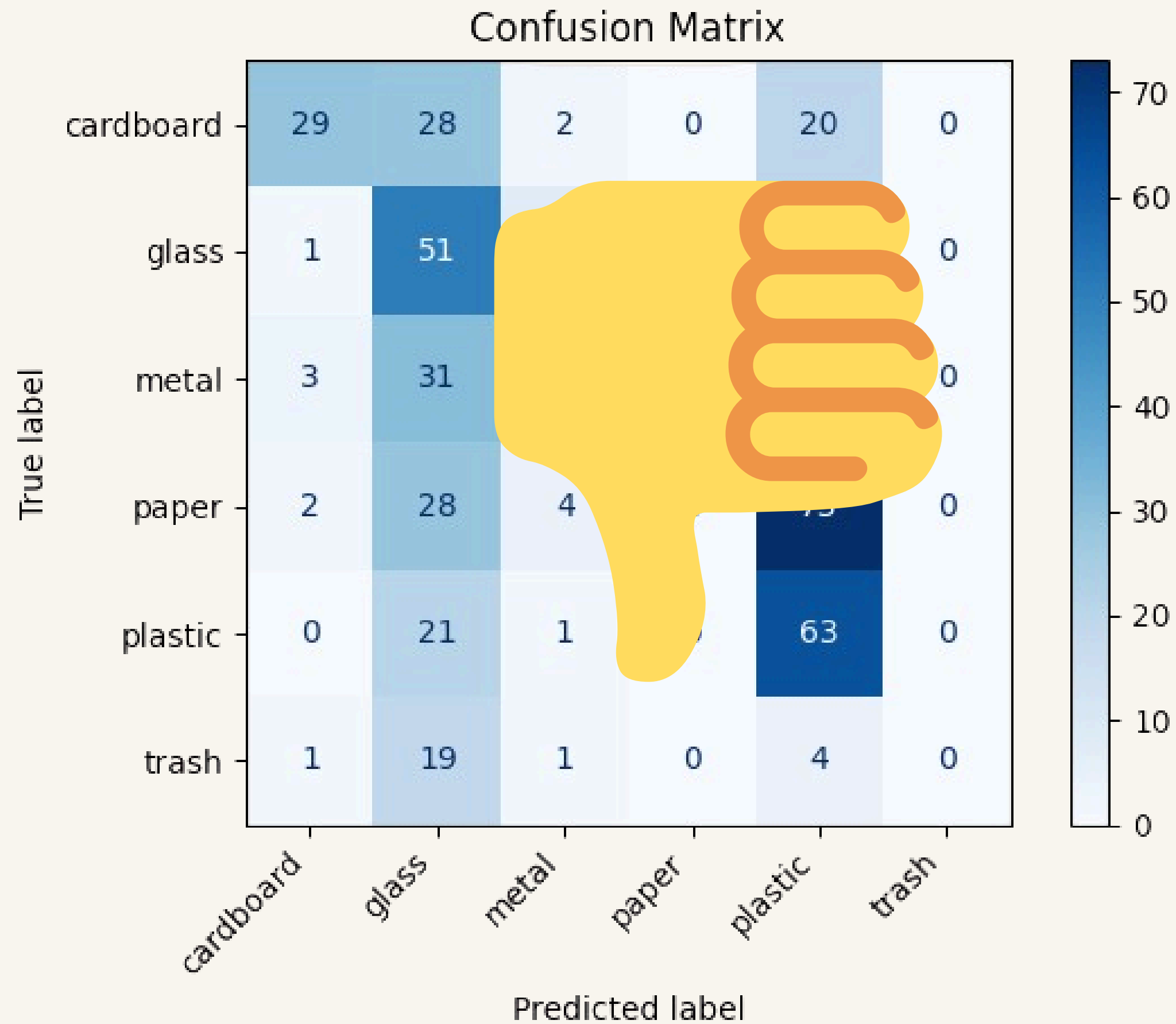
Class: metal



Class: paper



Multi layer Perceptron



CONVOLUTIONAL NEURAL NETWORK



- Looks at a group of pixels instead of individual pixels
- Can detect patterns such as edges and textures
- local patterns matter

```

class CNN(nn.Module):
    def __init__(self, num_classes=6):
        super(CNN, self).__init__()

        self.conv_layers = nn.Sequential(
            # Block 1: Input (3, 64, 64) Output (32, 32, 32)
            nn.Conv2d(in_channels=3, out_channels=32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2), # Reduces to 32 x32
            nn.Dropout2d(0.1),

            # Block 2: Input (32, 32, 32) Output (64, 16, 16)
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2), # Reduces to 16x16
            nn.Dropout2d(0.1),

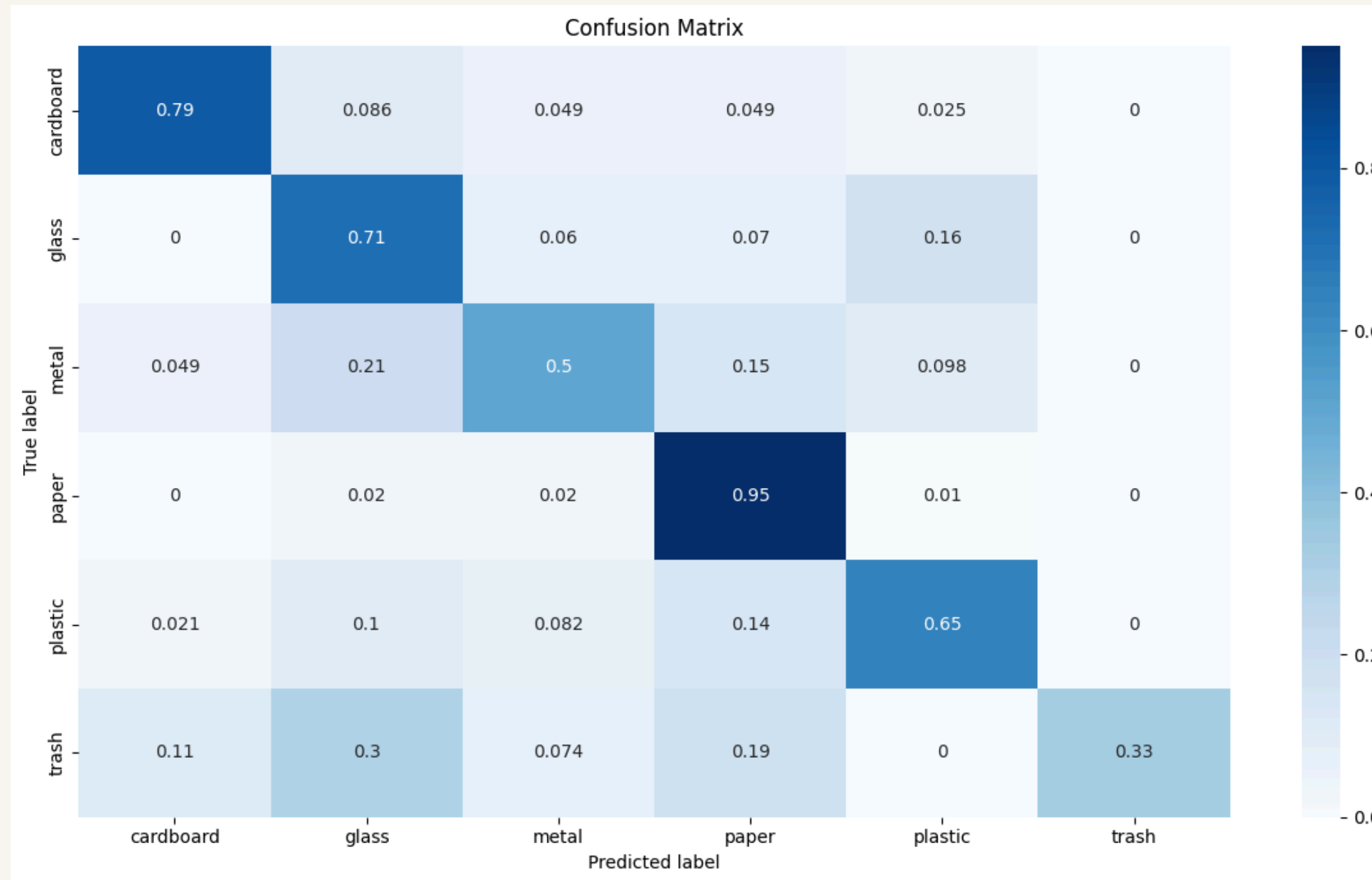
            # Block 3: Input (64, 16, 16) Output (128, 8, 8)
            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.MaxPool2d(2), # Reduces to 8x8
            nn.Dropout2d(0.2),

            # Block 4: Input (128, 8, 8) Output (256, 4, 4)
            nn.Conv2d(128, 256, kernel_size=3, padding=1),
            nn.BatchNorm2d(256),
            nn.ReLU(),
            nn.MaxPool2d(2), # Reduces to 4x4
            nn.Dropout2d(0.3),
        )

        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(256 * 4 * 4, 512)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.5)
        self.fc2 = nn.Linear(512, num_classes)

    def forward(self, x):
        x = self.conv_layers(x)
        x = self.flatten(x)
        x = self.fc1(x)
        x = self.relu(x)
        x = self.dropout(x)
        x = self.fc2(x)
        return x

```



71% accuracy

Future Improvements

- Larger/ better dataset
- more classes
- transfer learning
- Raspberry Pi

